OCP GLOBAL SUMMIT 2026 • TIME APPLIANCES TRACK

时间调度电路网络的预测误差预算

Sahas Munamala · Ahmad Byagowi

时长 14 分 52 秒

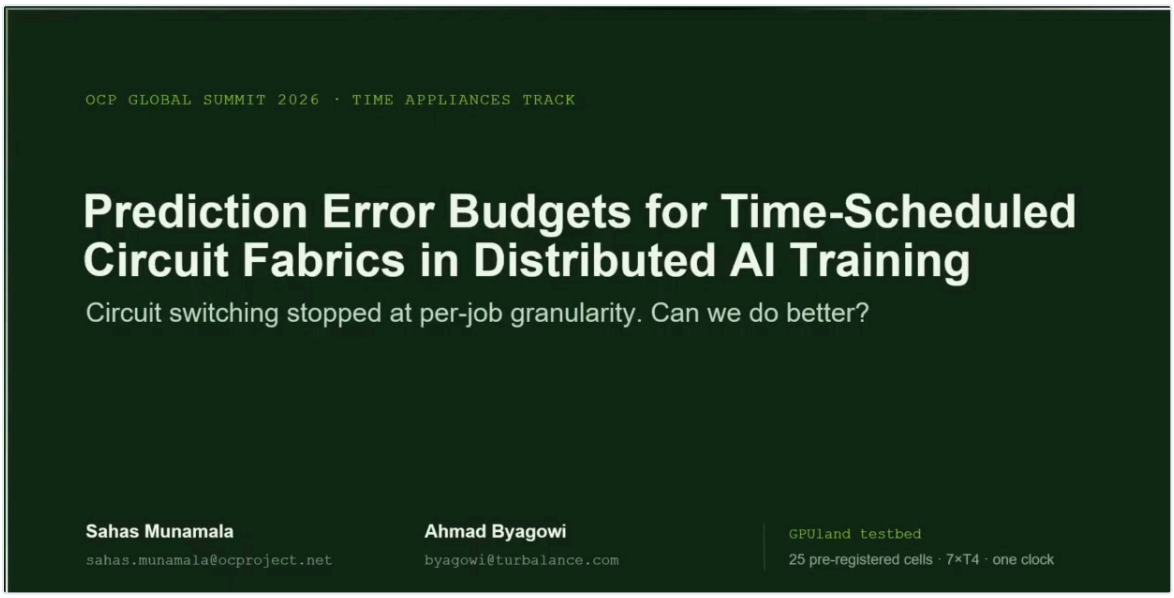
来源 youtube.com/watch?v=6nQAvEFFf6U

测试床 7×T4 · 25 个预注册单元 · 单一时钟

分布式 AI 训练中时间调度电路网络的预测误差预算

> OCP Global Summit 2026 · Time Appliances Track 讲者：Sahas Munamala（独立研究者）、Ahmad Byagowi（turbalance） 全长 14 分 52 秒 ｜ 共 13 页幻灯片 ｜ 测试床：7×T4，25 个预注册单元，单一时钟

01 开场：把时间用在 AI 训练的网络里



标题页：Prediction Error Budgets for Time-Scheduled Circuit Fabrics in Distributed AI Training，副标题为"电路交换停在每作业粒度，我们能做得更好吗？"；页脚注明 GPUland 测试床、25 个预注册单元、7×T4、单一时钟

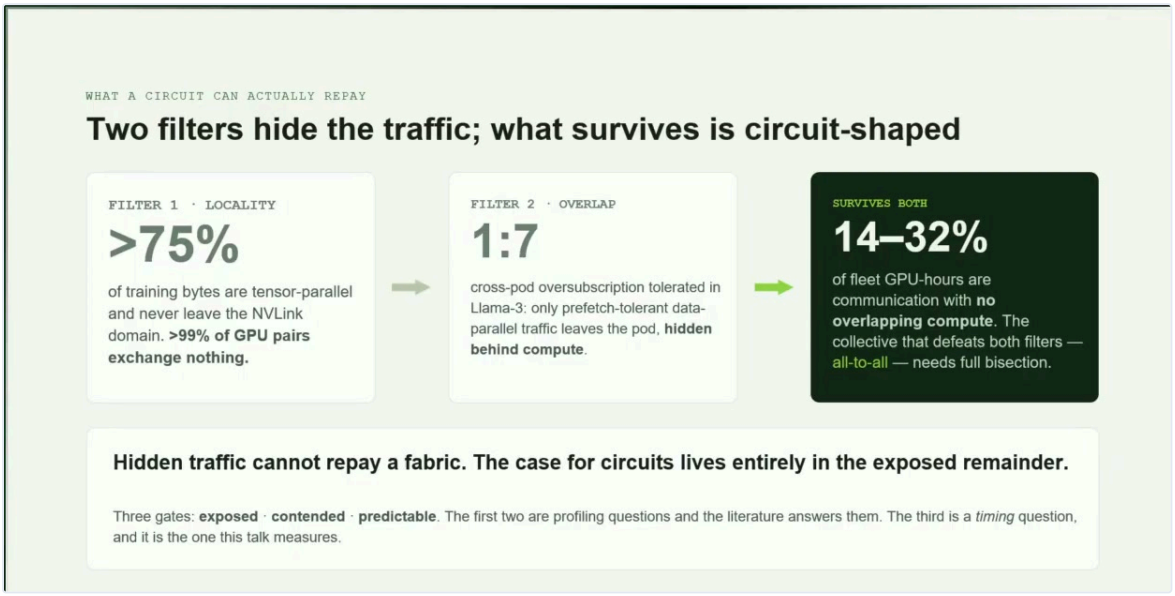
我们正在等幻灯片上传。好，来了。好，那么我再介绍一下 Sahas。呃，谢谢 Sahas，当然也谢谢 Ahmad 撰写这篇论文。呃，Sahas 是一位常驻圣何塞的独立研究者，即将加入 Everpure，他的工作聚焦于分布式 AI 训练网络的授时松弛（timing slack）与电路交换。

（Sahas） 是。呃，谢谢介绍。那么，呃，这项工作最初只是对电路交换及其如何有益于 AI 训练的一次探索，后来我意识到，呃，其中有一些关于同步的应用——尤其是训练轮次（training rounds）——可能会带来一些好处。所以，呃，我在这里会带大家走一遍我发现的若干东西，以及时间可能帮得上忙的地方。

> "We're just waiting for the slides to be uploaded. Okay, there we go. Okay, so I'll introduce Sahus again. Uh, thanks Sahus and of course Ahmed for authoring this paper. uh Sahus is an independent researcher based in San Jose soon joining Everpure and whose work focuses on timing slack and circuit switching for distributed AI training fabrics. Yeah. Uh thank you for the introduction. So um a lot of this work was inspired uh first as just an exploration of circuit switching and how it might benefit AI training and then I realized um there there's some applications of of uh synchronizing um especially like training rounds that um might provide some benefits. So um yeah uh here I'll kind of walk through some some things that I found and uh a place where timing might might help. though um I guess with"

> **小结**　研究起点不是"时间同步"，而是"电路交换能否受益于 AI 训练"；反过来才发现训练轮次的同步是一个可用的抓手。这一反转决定了全篇的论证方式：先量化网络侧的机会，再看时间在其中占多大分量。

02　两个过滤器：什么样的流量才是"电路形状"的



Two filters hide the traffic; what survives is circuit-shaped：过滤器一「局部性」——超过 75% 的训练字节是张量并行、从不离开 NVLink 域，99% 以上的 GPU 对不交换任何数据；过滤器二「重叠」——1:7 的跨 pod 超额订阅下，Llama-3 只有预取容忍的数据并行流量落在计算之后；两者都存活下来的仅 14–32% 的 GPU 小时，是无可重叠计算的暴露通信，而全对全集体需要全二分带宽

在 AI 训练的网络拥塞方面，有很多办法——就在它们组织计算的方式里——可以隐藏一部分拥塞及其影响。呃，第一个策略是把互相通信的东西放进同一个 pod，用高带宽连接。呃，而大多数消息实际上是在这些 pod 内部传输的。所以这捕获了大部分流量。呃，但显然有些东西需要离开这些 pod；Meta 报告的是 1:7 的超额订阅（oversubscription），意思是 pod 内的带宽是跨 pod 通信的七倍。所以对于 ring all-reduce 之类的东西、那些跨 pod 的东西，很多都被同时正在进行的计算隐藏掉了。

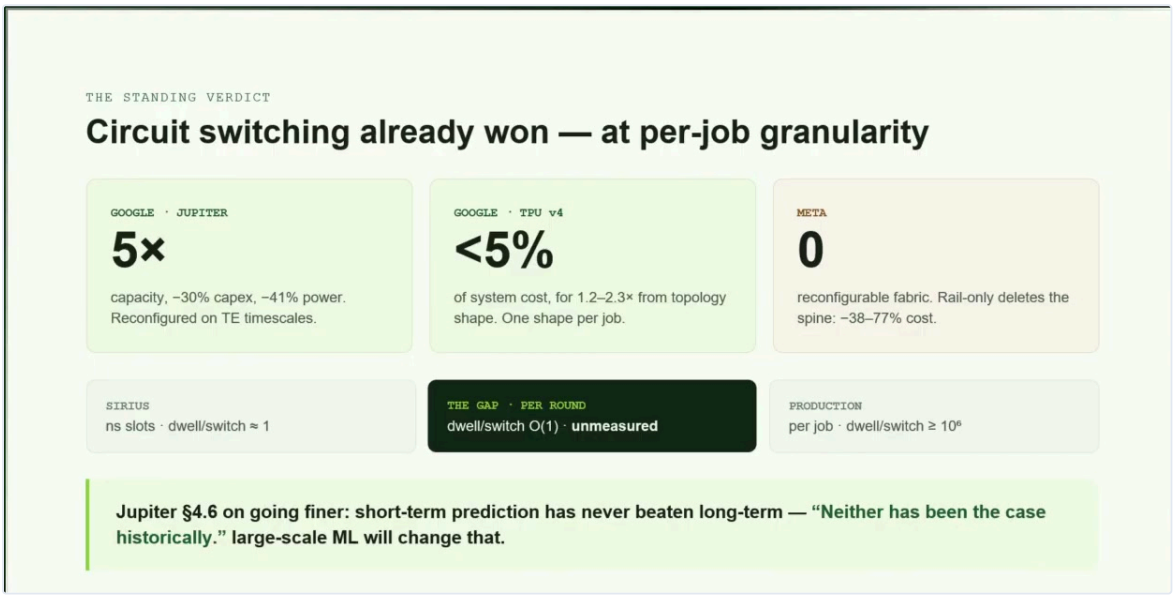
所以有一个概念叫"暴露通信"（exposed communication），指的是直接落在计算路径上、影响作业完成时间（JCT）的通信。呃，Meta 报告的是大约 14% 到 32% 的 GPU 小时是暴露的。这是"拥有一个巨型集群、同时跑着一大堆不同作业、以及掉队作业（straggler）"这些因素叠加的结果。所以，呃，在 AI 训练里基本上有非常可重复的通信模式，而这正是我们想用电路交换去利用的东西。

> "network congestion in AI training there's a lot of ways that um just in the way that they organize the compute they can hide some of the congestion and and and the effects of it. So um the the first strategy is uh putting things that communicate each other inside inside of a pod with with high bandwidth. Um and mostes that are transferred actually are are transferred within these pods. So that captures most most of the traffic. Um but there's obviously stuff that needs to leave these pods and meta um uh reports that a one to seven overs subscription meaning that um there's seven times the bandwidth within the pod versus uh cross pod uh communication. So uh for things like um like ring all reduce or or things like that um things that cross pods uh uh a lot of that is hidden by uh compute that's actually also happening at the same time. So there's a concept called exposed communication where um communication that is directly in the in the path of of comput um uh uh of job completion time. So um what we're uh what Meta is reporting is

> that it's around 14 to 32% of GPU hours is um is exposed. And this is a uh the combination of just having a giant fleet with a whole bunch of different kinds of jobs running at the same time and ideas of like straggling jobs and and things of that nature. So there's um um largely within AI training there's very repeatable uh communication patterns and that's what we're trying to exploit with with circuit switching."

小结　两道过滤器把"可被电路交换修复的流量"压到 14–32%：局部性（>75% 不出 NVLink 域，>99% 的 GPU 对不交换数据）与重叠（1:7 超额订阅下只有预取容忍的 DP 流量落在计算之后）。结论是"隐藏的流量救不了网络"——电路交换的价值完全在暴露的那部分，而全对全集体需要全二分带宽，正是电路的用武之地。

03　电路交换已经赢在"每作业"粒度



Circuit switching already won — at per-job granularity：Google Jupiter 以 5× 容量、约 −30% 资本开支、−41% 功耗换来按 TE 时间尺度重构；Google TPU v4 占系统成本 <5%，拓扑塑形带来 1.2–2.3×；Meta 可重构网络为 0（仅删除 spine，−38–77% 成本）；下方对比栏指出 SiriuS 的 ns 级时隙与驻留，而"每轮"这一粒度的驻留仍是 O(1)、未被测量

呃，Google 已经在他们的 AI 数据中心里用了电路交换。呃，他们展示的是：可以获得 5 倍的带宽容量，而总系统成本更低。呃，Meta 有他们自己的、仅限 rail 的版本，利用的是同一 rank 上大多数 GPU 都跨 pod 通信这一事实。呃，他们把这叫做 rail-only 架构。

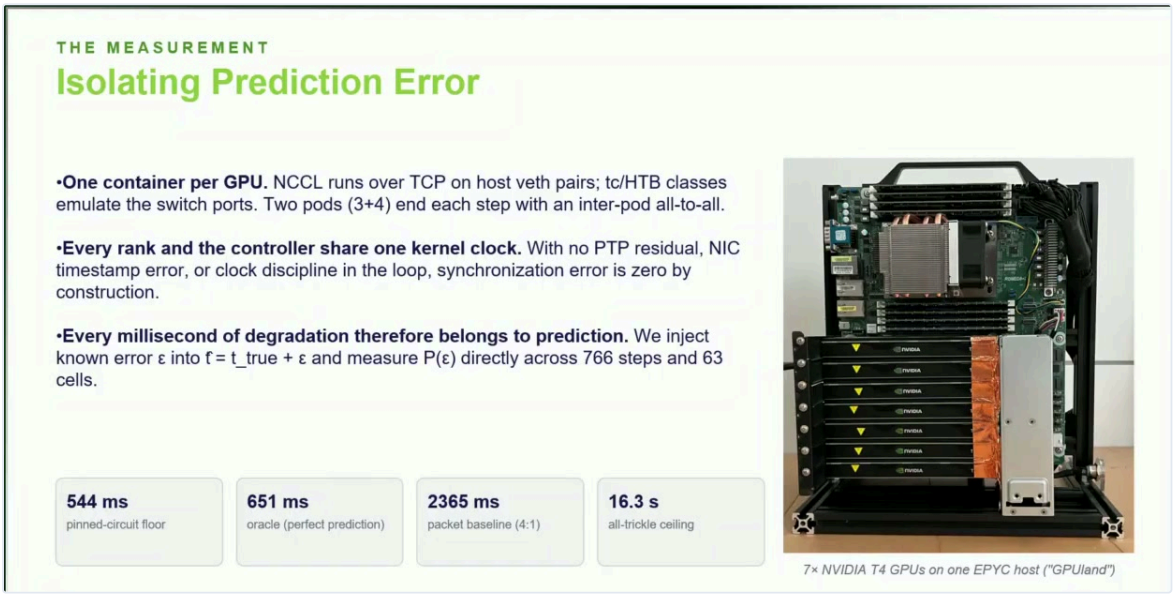
呃，电路交换有多个类别。第一类是纳秒级切换时间、用激光的那种；那种情况下他们基本上是在（多个目标）之间做轮转，把可用带宽切分给需要通信的东西。呃，但我们今天讲的是毫秒级的电路交换，大约在 10 毫秒这个量级。呃，目前 Google 是按作业重配，也就是说，对某个特定的 AI 训练作业，给它一个网络形状，让它在整个作业期间跑下去。但我想研究的是：我们能做得更好吗？能不能在轮次内切换？能不能利用其中一些并行性、拿到更好的结果？这就是我的实验设置……

> "So, uh, Google already has circuit switching within their, uh, AI data centers. Um, and what they're, uh, showing is that you can have 5x the bandwidth capacity, uh, for less total system costs. Um, Meta has their own, uh, rail only version of doing this, exploiting the fact that, um, most GPUs on the same rank communicate uh, across pods. So, um, they they call this a rail only architecture. Um there's uh like multiple classes of circuit switching. So the first is like these nancond um uh switching times with with lasers. Um and in which case they're kind of uh roundroining on uh

> across to partition the available bandwidth between things that need to communicate. Um but what we're talking about today is the millisecond class circuit switching which happens around like 10 10 milliseconds. So um uh currently Google is um they reconfigure per job uh meaning for a specific AI training job they they give it a network shape and let it run for that entire job. But um I wanted to investigate can can we do better? Can we switch within within around? Can we exploit some of the uh parallelism and and get better um um uh better results? So this is the uh setup that I had"

> **小结**　工业界证据：Jupiter 5× 容量、TPU v4 拓扑塑形 1.2–2.3×、Meta rail-only 删除 spine 省 38–77%。但这些收益都发生在"每作业"粒度。研究空白在于：把重配粒度从"每作业"推到"每轮"，需要 ms 级驻留——而这一档的驻留成本至今未被测量。

04　隔离预测误差：测量方法



Isolating Prediction Error：一个容器对应一块 GPU，NCCL 走 host 上的 TCP 对，用 tc/HTB 模拟交换机端口；两个 pod（3+4）每步以 pod 间全对全收尾；每个 rank 与控制器共享同一个内核时钟，没有 PTP 残留、NIC 时间戳误差或时钟纪律回路，因此同步误差按构造为零；注入已知误差 ε 使 f = t_true + ε，直接测量 P(ε)；底部四个基准：544ms 固定电路地板、651ms oracle（完美预测）、2365ms 分组基线（4:1）、16.3s 全涓流上限

……多谢 Ahmad。呃，这是七块 T4 GPU。每一块都在模拟一个 pod，而在内核内部你可以建立具有不同属性的不同网络。呃，所以我可以通过给每条链路设定带宽来模拟电路交换。呃，我们在这里看到的是：如果一切顺利，如果你能完美地把所有数据放进电路交换的轮次里，最小值是 544 毫秒——大概是这样，我给了它每秒 40 GB（电路开启时），以及每秒 4 兆字节——那只是涓流（trickle）流量，以防万一，而且你没法完美地估计电路什么时候需要切换。所以有一个 slack，还有一个 oracle。那加起来就是 651。

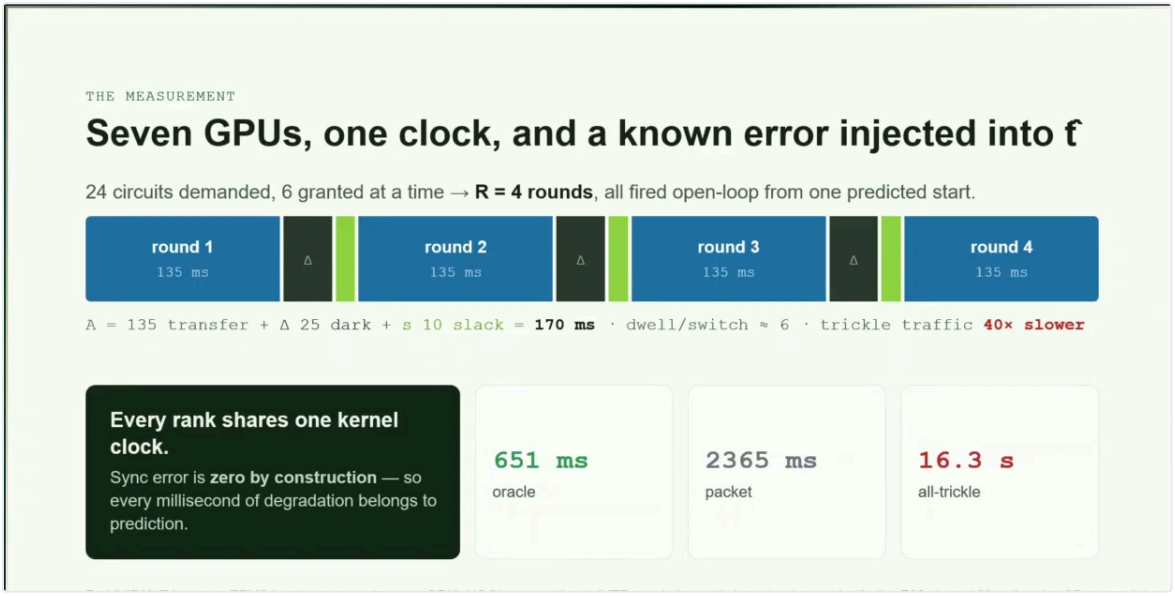
但呃，按 4:1 的超额订阅——也就是 pod 内带宽是 pod 外的四倍——电路……抱歉，分组交换的拥塞会把它推到接近两秒。呃，而 16.3 秒那个数字，来自轮次失败、整件事被卡在涓流流量上的情况。那么，这里发生了什么？我们有七块 GPU 需要彼此全对全通信，呃，那意味着 24 个直连……

> "thanks to Ahmed. Um so that's seven T4 GPUs. Each one of these is um emulating a pod and uh within the kernel itself you can um establish kind of different networks with different u uh properties. So uh I I can simulate a circuit switch by

> um uh giving each of these uh bandwidth links. And so um what we're seeing here is if everything goes right and if you uh perfectly uh um get all the data within the circuit switching rounds the minimum is 544 milliseconds that's kind of like um on my so I had I I gave it 40 gigabytes per second if if the circuit is on and I gave it uh 4 megabytes per second um that's like trickle traffic just uh in case and uh you can't have yeah you can't perfectly estimate when the circuit needs to switch. So there's a slack uh and a uh oracle. So that that adds the 651. But um with a 4:1 overs subscription meaning that there's four times more bandwidth within the pod than outside the pod, the the circuit uh sorry the packet switching congestion uh blows it up to almost two seconds. um and 16.3 seconds that number comes from if uh the the round fails and the the entire thing gets stuck through um the trickle traffic. So uh what what's going on here? So we there's seven GPUs that need to communicate with each other uh all to all and uh that means like 24 direct"

小结　方法的关键在于"把同步误差按构造归零"：所有 rank 与控制器共享同一个内核时钟，没有 PTP、没有 NIC 时间戳误差、没有时钟纪律回路。于是任何毫秒级劣化都只能归因于预测误差。四个基准把量纲钉死：544ms（电路地板）、651ms（完美预测）、2365ms（分组 4:1）、16.3s（全涓流上限）。

05　七块 GPU、一个时钟、一个已知误差



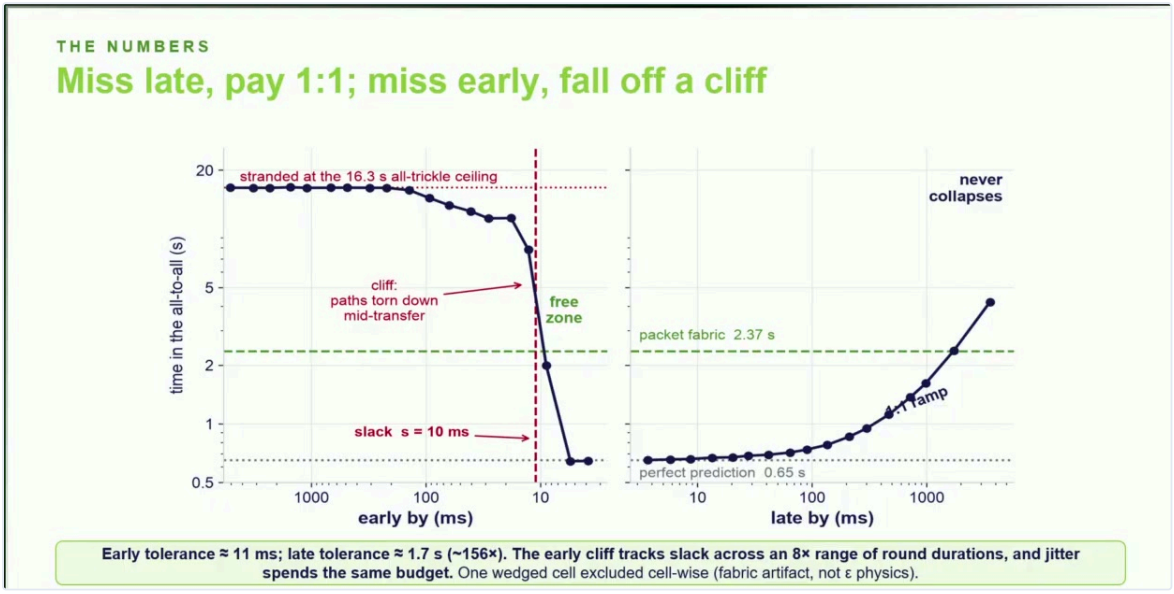
Seven GPUs, one clock, and a known error injected into f：24 条电路需求、每次只授予 6 条 → R = 4 轮，全部从同一个预测起点开环触发；每轮 135ms 传输 + Δ（25ms 暗时 + 10ms slack = 170ms），驻留/切换约 6ms，涓流流量慢 40×

……对（对等）连接，而电路交换一次只能服务六个。那么，我们能否不断地移动这些连接，去利用点对点链路的全部带宽、避开中间那个拥塞的 spine 或交换机？呃，这些数字和上一页是同一组。那么，当你预测得过早时会发生什么……

> "pairs and uh the circuit switch only serves six at a time. So can we uh kind of move uh can we uh continuously move the uh um the connections to exploit the full bandwidth of the pointto-point um and avoid this congested uh uh spine or switch in the middle uh and uh the these are the the same numbers from the previous slide. So, uh what happens when you predict"

> **小结**　测试拓扑可算：7 块 GPU 全对全 = 24 条电路，交换机一次只给 6 条，于是 R = 4 轮；每轮 135ms 传输加 170ms 切换开销，涓流比电路慢 40 倍。这个"6/24"的约束正是轮次必须被精确调度、也正因此误差会被放大成悬崖的原因。

06　晚了按 1:1 付费，早了掉下悬崖



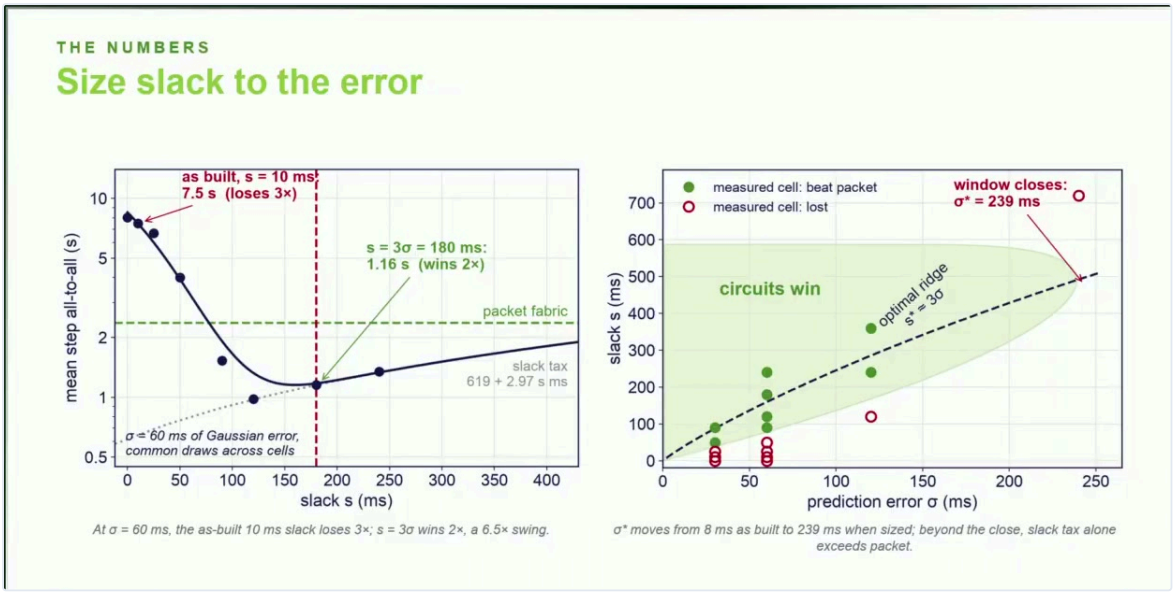
Miss late, pay 1:1; miss early, fall off a cliff：左图以"提前量"为横轴，显示提前过多会撞上 16.3s 全涓流天花板（中途拆路形成悬崖），落在 slack 内则回落到电路地板；右图以"滞后量"为横轴，劣化呈平缓 1:1 斜坡；页脚给出早期容差约 11ms、晚期容差约 1.7s（约 156×），早期悬崖在 8 倍轮次时长范围内跟随 slack

早，对吧？你会看到这里这个巨大的悬崖：如果你切换得太早，那么那一轮的剩余部分就得走涓流流量，没法继续下去。所以，你很容易就撞上那个天花板。但呃，如果你落在 slack 之内，它就会回落到电路地板。而如果你等得太久——也就是切换得太晚——你就开始失去相对于分组交换的优势。

> "early, right? You you'll see this big cliff here where if you uh uh if you switch too early, then the rest of that round needs to go on the trickle traffic and you can't um uh continue on. So, you you hit that you hit that ceiling very easily. But um if you're if you hit within the slack, then it drops down to the the circuit floor. But if you wait too long, uh meaning that if you switch too late, you you start losing your benefit compared to to packet switching."

> **小结**　误差的两个方向极不对称：早到超过容差就中途拆路、整轮掉进涓流，直接撞上 16.3s 天花板；晚到只是按 1:1 平缓付钱。实测早期容差约 11ms、晚期容差约 1.7s，相差约 156 倍。这条不对称性是全文的核心约束——预测系统必须"宁晚勿早"。

07　把 slack 按误差来定尺



Size slack to the error：左图显示按现状 s = 10ms 配置时损失 3×（7.5s），而 s = 3σ = 180ms 可赢得 2×（1.16s）；右图以预测误差 σ 为横轴，绿色区域为"电路获胜区"，最优点位于 σ ≈ 3σ，窗口在 σ* ≈ 239ms 处关闭

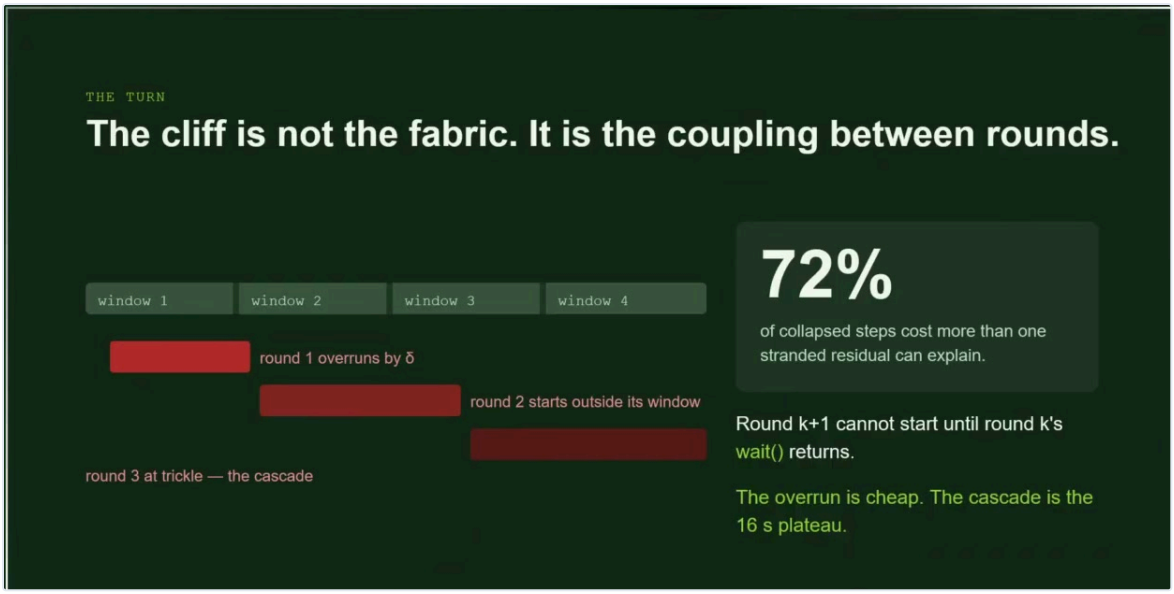
那么，对这一页我聚焦了一下——右边这张图，是基于预测误差（或者说这些轮次完成时刻的标准差），你还能不能赢过分组交换？那个绿色区域就是电路交换获胜的区域。呃，我聚焦在 60 毫秒标准差上，看左边这条收益曲线。如果你在那条绿线以下，那就是收益。

呃，所以我发现的是：你需要把 slack 定到大约是你误差标准差的三倍。呃，而目前报告里我们看到大约 400 毫秒的轮次时长波动，呃，这意味着你得把保护带设得相当宽。那么，有没有办法做得更好？现在……

> "And for so this I zeroed in on um uh uh like a so on the the right is a kind of graph of like based on the prediction error or standard deviation of like when do these rounds finish can you still beat packets? So that green area is the area where um circuit switching wins. Um, and I zeroed in on 60 uh millisecond standard deviation and kind of see the curve here on the left of of the benefit. So if you're below that green line, that's um a benefit. Uh so uh what I'm finding or what I found is you need to to slack uh size the slack to almost three times your um your uh standard deviation of error. And uh currently in uh reports we're seeing like 400 milliseconds of of round timing uh variation. Um which means that you'd have to uh set your your guard band pretty pretty wide. So um is there a way we can do better? So right now um the"

> **小结**　定尺规则：slack ≈ 3σ。按现状 s = 10ms 会损失 3×（7.5s），而 s = 3σ = 180ms 可赢得 2×（1.16s）。但现实中轮次时长波动约 400ms，保护带会被迫设得很宽——收益被自身的不确定性吃掉，这正是下一步要找"更好的办法"的动机。

08　悬崖不在网络，而在轮次之间的耦合



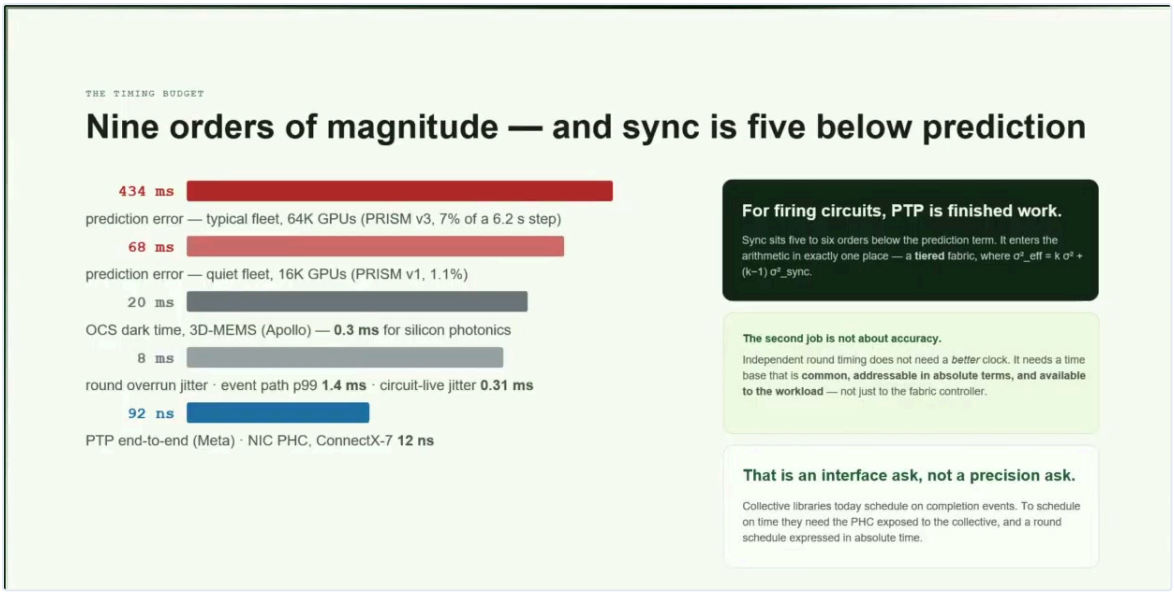
The cliff is not the fabric. It is the coupling between rounds：窗口 1–4 依次排开，轮次 1 超期 6、轮次 2 落到自己的窗口之外、轮次 3 开始涓流级联；72% 的崩塌步长所需的代价超过任何单个滞留残差能解释的量；轮次 k+1 必须等轮次 k 的 wait() 返回

……悬崖其实并不是网络无法服务它。而是每一轮都依赖上一轮完成。所以，呃，大多数撞到涓流地板的轮次，是因为那一轮在第一个窗口内没跑完。那么，影响我们时间松弛的来源有哪些……

> "cliff is not really that the fabric is um unable to surv um service it. it's that each round depends on the previous round completing. So, um, seven most of the stuff or or most of the rounds that ended at the uh trickle ceiling floor were because that uh uh the round wasn't complete during that first window. So, where does uh the the sources of like things"

> **小结**　关键诊断：悬崖是"轮次串行依赖"造成的级联，而非网络容量不足。轮次 k+1 必须等 k 的 wait() 返回，一次超期就把后续全部推出窗口。72% 的崩塌代价超出单个滞留残差可解释的范围——说明这是系统性放大，而不是偶发抖动。

09　九个数量级：同步比预测低五位



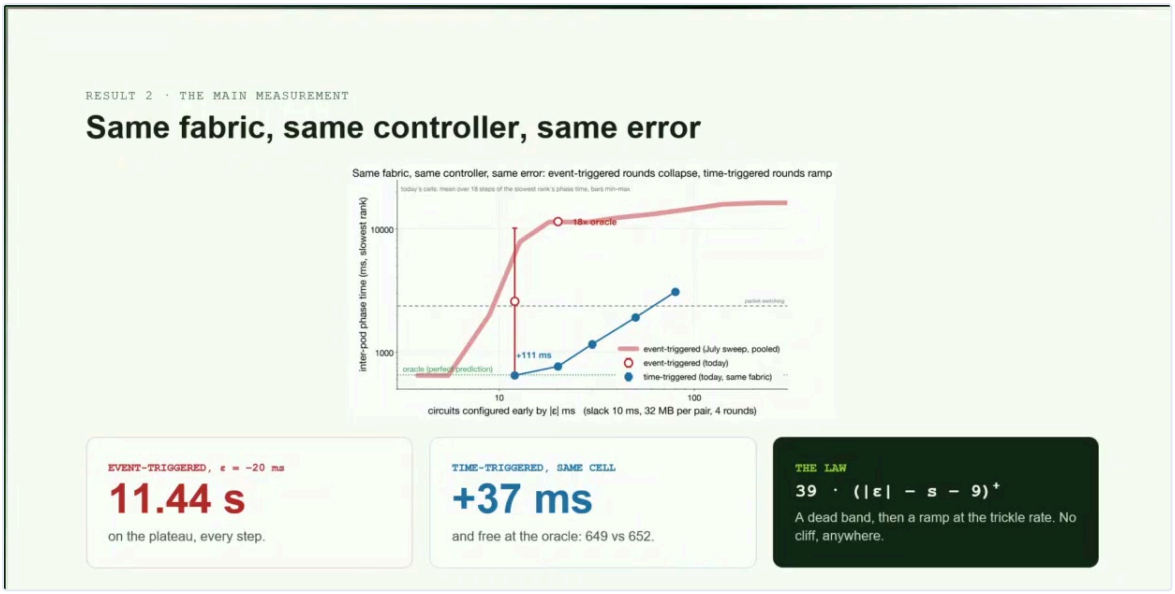
Nine orders of magnitude — and sync is five below prediction：预测误差 434ms（典型集群，64K GPU，PRISM v3 的 7%）、68ms（安静集群，16K GPU，v1 的 1.1%）、20ms（OCS 暗时，3D-MEMS/Apollo）、8ms（轮次超期抖动，事件路径 p99 1.4ms、电路级抖动 0.31ms）、92ns（PTP 端到端，Meta，NIC PHC、ConnectX-7 的 12ns）；右侧三块结论：对触发式电路，PTP 已是"完成的工作"；第二项工作不是精度而是接口；集体库需要的是按绝对时间表达的轮次表

……影响我们时间松弛的？它不是时间同步。那比我们谈的毫秒级要低好几个数量级。呃，OCS 切换在 20 毫秒量级，那仍然比会被叠加进来的预测误差低一个数量级。所以大部分其实是掉队的轮次时长。

> "that uh affect our timing slack? So it's not time synchronization. That's like orders of magnitude less than than the milliseconds we're talking about. Um the OCF switching is in in the 20 millisecond range which is still an order of magnitude less than um the the prediction error uh that we can u that gets added. So most of it is really the the straggling uh round timings."

小结　量级阶梯一目了然：预测误差 434ms（典型）→ 68ms（安静）→ OCS 暗时 20ms → 轮次抖动 8ms → PTP 端到端 92ns。同步比预测低五到六个数量级，因此"更准的时钟"不再是瓶颈；真正的缺口是接口——把 PHC 暴露给集体库，并把轮次表用绝对时间表达。

10　同一网络、同一控制器、同一误差：时间触发



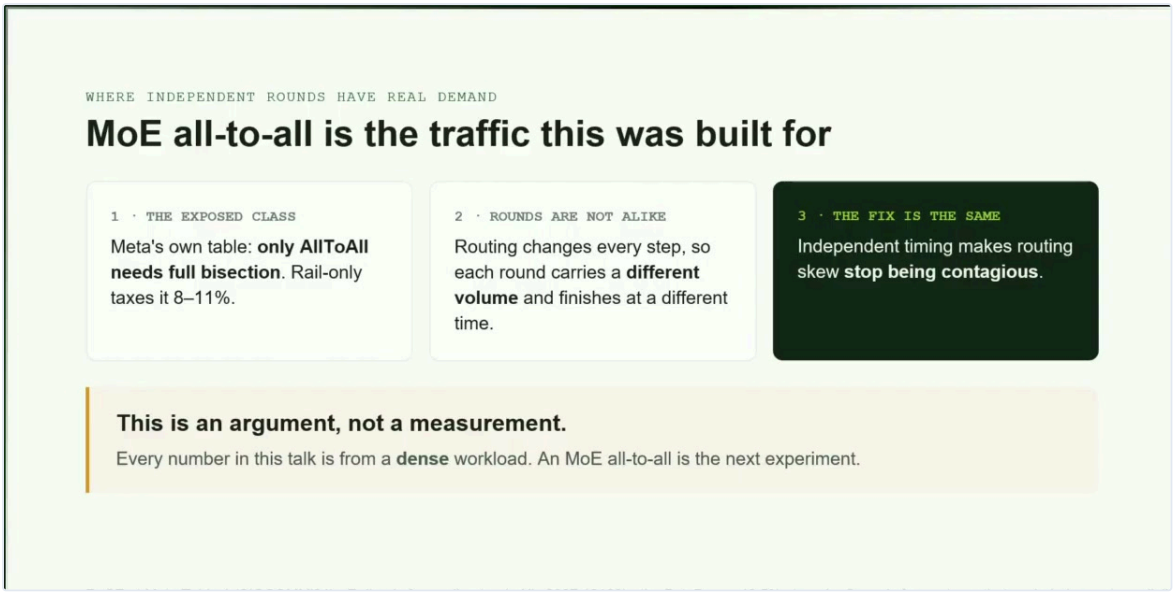
Same fabric, same controller, same error：同一单元里，事件触发（今天）在 ε = −20ms 时于平台上达 11.44s，而时间触发（同一网络）仅 +37ms，且在 oracle 处免费（649 对 652）；右下给出定律 $39\cdot(|\varepsilon| - s - 9)^+$——先是一个死区，然后按涓流速率爬坡，任何位置都没有悬崖

那么，如果轮次之间不互相依赖呢？实际会发生什么？呃，不再是那条蓝线所表示的悬崖——如果你能预先调度每一轮何时可以发生、并让之前的轮次承受涓流，你就可以得到那条……呃，等等，抱歉，红线才是悬崖，因为那条竖线是你加进去的 slack。但如果你能预先调度，那么就不再是悬崖，而是可以斜着切入——只要你在那条虚线以下，你就比分组交换更好。

> "So what if the the rounds weren't dependent on each other, right? What what actually happens? So uh instead of the cliff uh which is that blue line um if you can preschedu when each round uh can occur and let the previous rounds suffer the trickle uh you can instead have that um uh that wait yeah the um sorry the red line is the cliff because that uh that vertical line is is the slack that you've you've added. But if you um if you're able to to preschedu instead of having that cliff, you can you can slope in uh where if you're below that dotted line, you're you're better than than packet switching."

小结　同样的硬件、同样的控制器、同样的误差，只因触发方式不同结果天差地别：事件触发在 ε = −20ms 时整段卡在 11.44s 平台；时间触发仅 +37ms，且在 oracle 点几乎免费（649 vs 652）。经验定律 $39\cdot(|\varepsilon| - s - 9)^+$ 描述的是"死区 + 涓流速率爬坡"，没有悬崖——这就是把"同步"变成"调度"的收益。

11　MoE 全对全：这套机制真正为之而生的流量



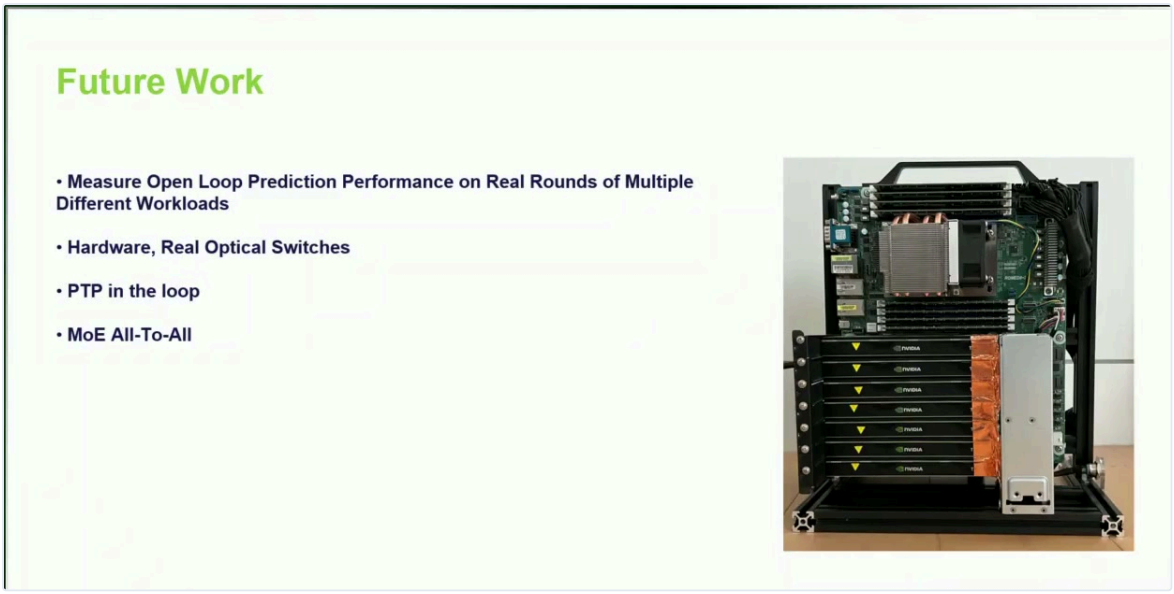
MoE all-to-all is the traffic this was built for：①暴露类——Meta 自己的表格显示只有 AllToAll 需要全二分带宽，仅 rail 会带来 8–11% 的代价；②轮次并不相同——路由每步都在变，因此每轮承载不同体量并在不同时刻结束；③解法相同——独立授时让路由偏斜不再具有传染性；页脚明确"这是论证，不是测量"

那么，什么样的流量才是独立的全对全？那就是混合专家（MoE）：你可能有四个专家同时激活，你需要在你所有的专家之间做一次全对全集体通信。呃，这件事还在进行中，还需要工作才能拿到真实数字。呃，那么未来的工作……

> "So what kind of traffic actually is independent all to all? So that's mixture of experts where you have multiple um experts maybe four that are active and you need to um uh basically uh do an alltoall collective across all your uh all your experts. So um this is something that uh is still in progress still needed work um uh uh to get actual numbers. Uh so uh future work for uh"

> **小结**　MoE 与本文假设天然契合：只有 AllToAll 需要全二分（rail-only 要付 8–11% 代价）；路由每步变化使各轮体量与时点都不同，因而"独立授时"不会传染偏斜。作者明确标注这只是论证而非测量——真实数字仍待硬件验证。

12　未来工作



Future Work：在多种不同工作负载的真实轮次上测量开环预测性能；真实硬件与真实光交换机；把 PTP 放进回路；MoE 全对全

……接下来会发生什么。呃，首先是拿到真实的预测性能误差，并在不同规模的多种工作负载上分析它们。呃，我非常希望能用上真实硬件的光交换机，这样就能做真实硬件分析。如果我能进入训练轮次、为混合专家真正预先调度每一轮何时发生——我们现在大概顶到 50 个专家，那对全对全来说几乎是完美的——那么，如果我们能做成这种轮转，我或许就能拿到真实数字展示给大家。呃，非常感谢各位聆听。呃，我只想说，谢谢 Ahmad 给我硬件、给我这次机会；这个社区对我一直很欢迎，这大概是我在这里的第一次演讲。所以……

> "what's going on. Uh so first is getting actual prediction performance errors and uh analyzing those on different workloads of different sizes. Uh I I would love to have access to real hardware optical switches so I can do um uh real hardware analysis and if I could uh uh um get into the the training rounds and actually preschedu when each round happens for mixture of experts which actually is the uh we're kind of topping out at like 50 experts um which is uh almost perfect for for all to all. So, um if we can uh kind of do this round robining thing, we uh I might be able to to get some real numbers and and show and show you guys. So, uh thank you so much for for listening. Uh I just want to say uh thank you to Ahmed for for giving me the hardware and uh this time in community has been this is my uh I guess first uh presentation here. So, um"

小结　四条后续路径：真实负载的预测误差测量、真实光交换机硬件、把 PTP 纳入回路、以及 MoE 全对全的预调度。其中"PTP 进回路"意味着从"假设同步免费"走向"显式测量同步贡献"——这是本文结论的下一步验证。

13　致谢与问答



Thank You 页：Prediction Error Budgets for Time-Scheduled Circuit Fabrics in Distributed AI Training，Sahas Munamala 与 Ahmad Byagowi，附 GPUland 测试床信息

……是的，这一直很友好，希望下次还能见到各位。（掌声）好，我们把时间交给提问。也许我可以先开个头：如果你能再解释一下那个悬崖、那个倒过来的图，会很有帮助。对，左边那张。

（讲者） 哦，这张。对。呃，是的。这是针对全对全轮次的——那条竖线是平均步进时间。所以如果你没有足够的 slack，你只是——怎么说呢——对，你会在总作业完成时间上损失。呃，这只是扫描你需要多少 slack，而它交汇的那个点大约是轮次时长标准差的三倍。

（提问者） 那个 slack 税——slack tax。

（讲者） 对，我是说，如果你给系统加 slack，就会有一个最小量的作业完成时间被加上去，它几乎就是一条保护带。

（主持人） 好，还有别的问题吗？别告诉我这讲得超级清楚。我们没有问题。

> "yeah, it's been it's been inviting uh and I hope to see you guys next time. [applause] All right, let's open the floor for questions. Maybe I can start. It will be great if you can explain a little bit more. The one with the cliff that the inverted one. Yeah, the one on the left. Oh, this one. Yeah. Uh so, uh yeah. So for uh so this is for the all to all rounds where uh if you are so that vertical is the the mean step time. So if you don't have enough slack, uh you just kind of uh uh if you don't have enough slack, you just uh what do you call um yeah, you you you lose time on on total job completion. So um this is just for just sweeping uh the total uh how much slack you need to have and that point where it meets is about three times u the the standard deviation of the the round timings. The slack tax slack tax. yeah uh so I mean there's a minimum amount of uh job completion time that's uh added if you add Slack to the system where the it's just a guard band almost. Yeah. All right. Any more questions? Don't tell me this was super clear. We don't have any questions."

> **小结**　问答补充了两点澄清：悬崖图的竖线是平均步进时间，交汇点约为轮次时长标准差的 3 倍；"slack tax"指为系统加 slack 所必然带来的最小作业完成时间增量——本质上就是保护带的代价。这也是全篇的取舍：用可控的保护带换取消除悬崖。

附录　时间轴

时间	内容
00:15	开场与研究缘起
01:03	两道过滤器：局部性与重叠，暴露通信 14–32%
03:00	电路交换的工业界证据；从"每作业"到"每轮"
04:36	测试床与方法：7×T4、单一时钟、误差注入
06:18	24 条电路 / 每次 6 条 = 4 轮
07:03	悬崖：早到撞天花板，晚到 1:1 付钱
07:45	把 slack 定到 3σ
08:54	悬崖源于轮次耦合
09:21	九个数量级：同步比预测低五位
09:54	时间触发的定律 39·(
10:45	MoE 全对全
11:12	未来工作
12:06	致谢与问答
14:52	结束